



Explainable AI model for PDFMal detection based on gradient boosting model

Mona Elattar^{1,2} · Ahmed Younes² · Ibrahim Gad¹ · Islam Elkabani^{2,3}

Received: 7 October 2023 / Accepted: 29 July 2024 / Published online: 5 September 2024
© The Author(s) 2024

Abstract

Portable document formats (PDFs) are widely used for document exchange due to their widespread usage and versatility. However, PDFs are highly vulnerable to malware attacks, which pose significant security risks. Existing defense mechanisms often struggle to effectively detect and mitigate these threats, highlighting the need for more robust solutions. This paper introduces a robust framework that uses advanced tree-based ensemble models to detect malicious PDFs using the Evasive-PDFMal2022 dataset. The proposed model achieves a recall rate of 100%, an accuracy rate of 99.95%, and a fast inference time of 0.1723 s. Furthermore, the framework exhibits minimal false positive and false negative rates, ensuring a high level of precision in distinguishing between malicious and benign PDFs. Shapley additive explanations are used to improve the interpretability and reliability of the model's predictions. The results highlight the effectiveness of the proposed model in improving PDF document security and addressing the challenges posed by malware attacks.

Keywords Malware detection · Malicious PDF · Tree-based ensemble models · Explainable artificial intelligence (XAI)

1 Introduction

PDF malware is a growing concern, with recent reports indicating a significant increase in attacks targeting this widely used file format. This alarming trend underscores the urgent need for advanced solutions to detect and mitigate these sophisticated threats effectively.

PDFs are the standard format for digital documents and a common target for attackers who exploit vulnerabilities

to inject malware [1–5]. The versatility of PDFs, which can contain text, images, hyperlinks, rich media, and embedded code, makes them ideal for such exploits. Attackers frequently target outdated PDF readers or browser-based engines, injecting scripts directly into system memory and leaving no trace on the infected system [2, 4, 5]. In some cases, they use scripting tools embedded in Adobe Reader, such as AcroForms or XFA Forms, to inject scripts directly into the CPU's main memory, bypassing local file systems [5].

Malware designed to compromise data and systems can take various forms, including trojans, viruses, spyware, and ransomware [6]. PDF files are particularly susceptible to malware, which can spread through malicious attachments, downloads, and phishing campaigns. Attackers often exploit outdated PDF readers, highlighting the critical importance of keeping software up to date.

Effective malware detection requires a comprehensive understanding of malware operations. Traditional detection methods, such as signature-based and behavior-based techniques, often require expert analysts and struggle to adapt to evasive attacks and evolving malware variants [1, 7–9]. Signature-based detection identifies malware by unique identifiers, like specific byte sequences or hash

✉ Ibrahim Gad
ibrahim.gad@science.tanta.edu.eg

Mona Elattar
mona.abdelazim@science.tanta.edu.eg

Ahmed Younes
ayounes@alexu.edu.eg

Islam Elkabani
islam.Kabani@alexu.edu.eg

¹ Department of Computer Science, Faculty of Science, Tanta University, Tanta 31527, Egypt

² Department of Mathematics and Computer Science, Faculty of Science, Alexandria University, Alexandria 21511, Egypt

³ Faculty of Computer Science and Engineering, Al Alamein International University, Alamein 51718, Egypt

values, associated with known threats [8]. Behavior-based detection evaluates an object's intended actions by observing its behavior in a secure sandbox or isolated environment before execution [7, 9]. Heuristic-based techniques, utilizing artificial intelligence, offer a more adaptable solution by analyzing program activity to differentiate between legitimate and malicious behavior [8, 10].

Machine learning (ML) models, often employing heuristic methods, can analyze PDF files to assess their maliciousness. ML's ability to process vast amounts of data and generate rapid predictions makes it a valuable tool for malware detection. However, the effectiveness of ML models heavily depends on the size and quality of the training data. Both the quantity and quality of data are crucial for developing accurate and reliable models that generalize well to unseen datasets [11, 12].

While deep learning (DL) has shown promise in areas such as text, video, and audio analysis, its superiority over ML for tabular data remains uncertain. DL algorithms are typically computationally intensive, require larger datasets for training, and can be time-consuming [13].

Ensemble learning is an advanced machine learning technique that combines multiple models to increase overall performance. Tree-based ensembles, such as bagging and boosting, have proven particularly effective for handling tabular datasets. Their efficiency and accuracy have made them popular for various cybersecurity applications, including malware detection, software defect prediction, and fraud detection [11, 14–19].

Current PDF malware classification algorithms face several critical challenges. Inadequate data preparation can significantly reduce detection accuracy. Manual feature engineering is labor-intensive and error-prone. Models often struggle with advanced obfuscation and evasion techniques. Performance limitations, such as slow detection speeds and resource-intensive methods, hinder real-time applications. Balancing false positives and false negatives is crucial for effective security. Additionally, explainable models are essential for user trust and debugging. These shortcomings reduce the effectiveness of current security solutions, leaving systems vulnerable to infection, with potentially severe consequences, including data breaches, and financial losses.

This paper introduces a novel framework that addresses these challenges by leveraging high-performance tree-based models [11], incorporating effective data preparation techniques, and employing explainable AI methods. The proposed approach significantly improves detection accuracy, minimizes false negatives and false positives, and enhances model interpretability, fostering greater trust in the predictions. By tackling these critical issues, the

framework offers a more reliable and robust solution to the growing threat of PDF malware.

The main contributions of the study are as follows:

- The PDF malware detection framework shows significant enhancements in performance metrics and detection speeds, effectively reducing the occurrence of false negatives and false positives.
- The advanced feature engineering approaches, such as CatBoost encoding, Interquartile Scaler, and SMOTE, are used to improve model performance and robustness.
- The SHAP technique is used to provide useful insights into the decision-making process of the model.
- The Mann–Whitney U test and Wilcoxon signed-rank test are used to determine whether there are statistically significant differences in the performance metrics between the 2 sets of models.

The remaining sections of this paper are organized as follows: Sect. 2 provides an overview of relevant studies in the field. Section 3 outlines the dataset details, describes the employed feature engineering techniques, and discusses the tree-based ensemble models utilized in the research. Section 4 presents an analysis of the model performance, compares the obtained results with existing methods, incorporates Explainable AI (XAI) techniques, and employs statistical tests to assess the significance of the findings. Section 5 presents the results in comparison with the state-of-the-art techniques, and the limitations of the proposed work. Finally, Sect. 6 summarizes the key findings, highlights the research's significance, and suggests potential future directions.

2 Related work

Research in PDF malware detection has extensively explored ML and DL techniques, often utilizing datasets such as Contagio and Evasive-PDFMal2022 [20, 21]. These datasets provide a foundation for developing and testing new methodologies in malware detection.

Ahmed et al. [22] conducted a comparative analysis of PDFID and PeePDF tools, integrating an ML model to derive a novel feature set that enhanced PDF malware detection. Using random forest for feature importance assessment significantly improved classification performance and reduced training time. Similarly, Yoo et al. [23] introduced AI-HydR, a hybrid model that combines random forest with deep learning techniques, featuring 12 hidden layers for malware classification.

Lu et al. [24] developed a framework for detecting malicious documents across various formats, including PDF and Microsoft word. Their approach employs text parsing tools and the TF-IDF algorithm for feature

extraction and selection. While achieving detection times comparable to antivirus tools like Kaspersky, their framework demonstrated significantly higher accuracy.

Issakhani et al. [3] presented the Evasive-PDFMal2022 dataset, addressing the limitations of the Contagio dataset and establishing it as the most evasive dataset in the field. They utilized PyMuPDF and PDFID tools to develop tabular features and proposed a stacking technique for PDF malware detection. This method combines SVM, random forest, MLP, and AdaBoost learners, with logistic regression as the meta-learner, achieving an accuracy of 98.69% and an F1-score of 98.77%.

Abu Al-Haija et al. [13] employed an 80/20 split of the Evasive-PDFMal2022 dataset for training and testing. Their AdaBoost model, trained with 5-fold cross-validation, achieved an accuracy of 98.84% and an F1-score of 98.80%. Ekholm [25] proposed a 3-detector approach, with each detector using a random forest classifier trained on different datasets: Contagio, Evasive-PDFMal2022, and a combination of both. The detector trained on the Evasive-PDFMal2022 dataset achieved an accuracy and F1-score of 98.90% when tested on unseen data.

Chandran et al. [26] developed the IWO-S-LSTM model, a deep learning approach incorporating categorical encoding, null value removal, and outlier detection. This model features 3 hidden layers of Stacked LSTM (S-LSTM) and is fine-tuned using the invasive weed optimization (IWO) algorithm. It achieved an accuracy of 98.20% and an F1-score of 96.40% on the Evasive-PDFMal2022 dataset.

3 Methodology

This section describes the proposed framework for PDF malware detection, as illustrated in Fig. 1. The framework has 3 main stages: data description, feature engineering, and tree-based ensemble models with parameter optimization.

3.1 Dataset description

The Evasive-PDFMal2022 dataset, an enhancement of the Contagio dataset, consolidates information from both VirusTotal and Contagio to address challenges such as duplicate entries, incomplete coverage, and data biases [3]. Comprising 10,025 samples, this dataset offers a more inclusive and authentic portrayal of real-world PDF malware. Each sample is characterized by 31 static features (11 general and 20 structural) and is categorized as either malware or benign. Evasive-PDFMal2022 was utilized for the training and testing phases. During the validation phase, the model is tested using new data collected from

diverse sources. The provided examples consist of 1000 PDF files that are benign and 55 PDF files that are malicious.

3.2 Feature engineering

Real-world data is inherently noisy, so the initial and crucial step involves thorough data analysis to ensure consistency and remove inaccuracies.

3.2.1 Data analysis and preparation

The Evasive-PDFMal2022 dataset is partitioned into an 80/20 split for training and testing purposes. Data analysis encompasses the elimination of null values through the Listwise elimination method [27] and rectification of inconsistencies. For instance, in cases where special characters are detected, the “JS” feature is assigned a value of zero. Additionally, the “header” feature, denoting the file format and version, is standardized to the appropriate format (“%PDF 1:x” where x varies from 0 to 7) [1].

3.2.2 Categorical variables encoding

Two encoding methods were used: label encoding for the target variable and CatBoost encoding for categorical features. CatBoost encoding, which takes into account both the feature and the target, is chosen for its ability to achieve high model performance [28].

3.2.3 Interquartile scaler

The Interquartile scaler [24], which is robust against outliers, scales attribute values to a uniform range by removing the median value and scaling the data based on the interquartile range (IQR) between the 20th and 50th percentiles. Equation 1 presents the formula for the Interquartile scaler:

$$InterquartileScaler = \frac{X(i) - median(x)}{Q3(x) - Q1(x)} \quad (1)$$

where $X(i)$ is the value of the i th data point, $median(x)$ is the median of the data, $Q1(x)$ is the first quartile (20th percentile) of the data, and $Q3(x)$ is the third quartile (50th percentile) of the data.

3.2.4 Synthetic minority oversampling technique (SMOTE)

SMOTE tackles class imbalance by producing synthetic samples for the minority class, thereby preventing overfitting. Rather than duplicating existing instances, SMOTE functions within the feature space, generating new instances through interpolation between nearby positive

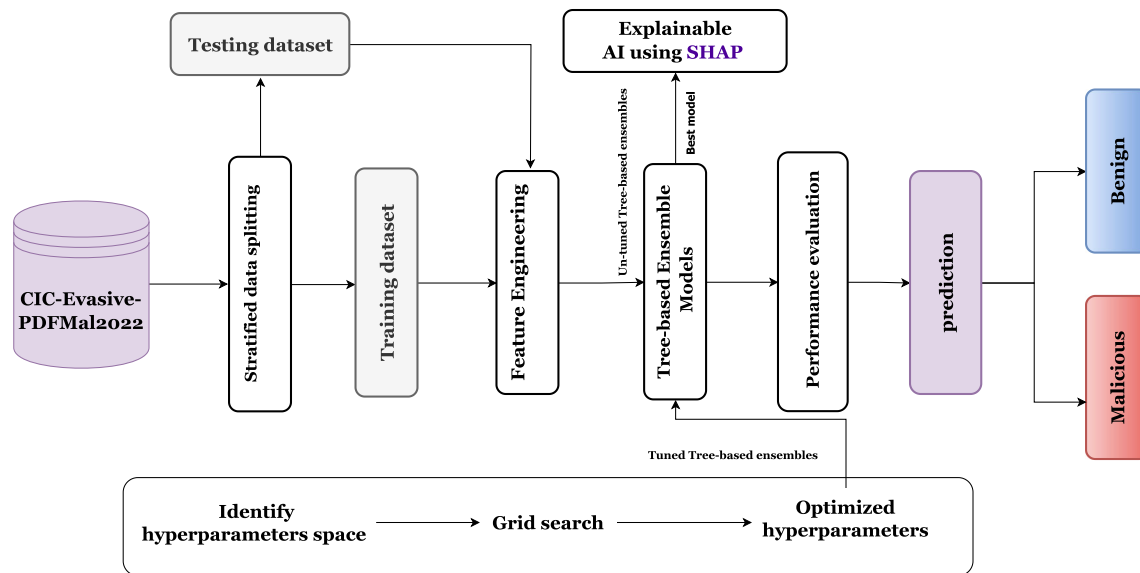


Fig. 1 The framework of the proposed approach for Evasive PDF malware detection

instances [29]. This method enhances the dataset and promotes a more balanced representation of the minority class.

3.3 Tree-based ensemble models

The proposed approach utilizes decision tree (DT)-based models exclusively. Tree-based ensemble models often have numerous parameters that significantly impact their performance. Grid search is employed to assess model generalization and optimize parameters [30]. The following sections describe the tree-based ensembles selected for this study.

3.3.1 Random forest (RF)

Random forest represents a sophisticated ensemble learning method that amalgamates numerous decision trees to elevate model efficacy. This technique mitigates overfitting by training each tree on a randomized subset of the data and features. As a result, random forests foster diversity among the individual trees, curbing the reliance on particular feature combinations [31]. Ultimately, the collective prediction is derived by aggregating the forecasts of all constituent trees, commonly achieved through majority voting or averaging. The refined parameters employed for the random forest classifier are detailed in Table 1.

3.3.2 Extra Trees (ET)

Extra Trees is an ensemble learning approach that randomly selects split thresholds for each feature and grows trees using the entire training dataset [32]. As a result,

Table 1 Tuned parameters for random forest classifier

Parameter	Search space	The best values
max_features	[auto, sqrt, log2]	sqrt
n_estimators	[50, 100, 200, 500]	50
max_depth	[3, 4, 5, 7]	7
min_samples_split	[2, 5, 10]	2
min_samples_leaf	[1, 4, 6, 8]	1

Extra Trees encourages diversity among the trees and can potentially reduce overfitting. The refined parameters employed for the Extra Trees classifier are detailed in Table 2.

3.3.3 Histogram-based gradient boosting (HGB)

HGB is an ensemble learning technique that accelerates both training and inference by binning feature values into histograms at each leaf node of the decision tree. As a

Table 2 Tuned parameters for Extra Trees classifier

Parameter	Search space	The best values
max_leaf_nodes	[2, 5, 8, 13, 21, 34]	34
max_features	[auto, sqrt, log2]	sqrt
max_depth	[2, 5, 8, 13, 21]	15
min_samples_split	[2, 5, 8, 13]	2
n_estimators	[20, 50, 100, 200]	50

consequence, HGB is effectively reducing the number of features considered [33]. The refined parameters employed for the histogram-based gradient boosting classifier are detailed in Table 3.

3.3.4 Adaptive boosting (AdaBoost)

AdaBoost is an ensemble learning approach that combines multiple weak learners, often decision stumps, into a strong learner by iteratively focusing on misclassified instances. This approach assigns higher weights to these challenging examples, forcing subsequent learners to pay more attention. The final prediction is a weighted majority vote, where each learner's performance determines its weight. As a consequence, AdaBoost improves accuracy and handles difficult cases more effectively [34]. The refined parameters employed for the AdaBoost classifier are detailed in Table 4.

3.3.5 Categorical boosting (CatBoost)

CatBoost is an ensemble boosting algorithm that uses symmetric trees with the same split at each level. This technique effectively handles categorical features and missing values without preprocessing [35]. As a result, CatBoost reduces the need for extensive hyperparameter tuning and mitigates overfitting, resulting in more generalized models. The refined parameters employed for the CatBoost classifier are detailed in Table 5.

3.3.6 Extreme gradient boosting (XGBoost)

XGBoost is an ensemble learning approach that builds decision trees in parallel. This parallel approach significantly improves efficiency and scalability, particularly for large datasets. XGBoost's level-wise strategy evaluates potential split points at each level of the tree using gradient values and partial sums. As a consequence, XGBoost helps find optimal splits and build more accurate models [36]. The refined parameters employed for the XGBoost classifier are detailed in Table 6.

Table 4 Tuned parameters for adaptive boosting classifier

Parameter	Search space	The best values
algorithm	[SAMME, SAMME.R]	SAMME
n_estimators	[5, 9, 10, 20]	20
learning_rate	[0.02, 0.03, 0.04]	0.04

Table 5 Tuned parameters for categorical boosting classifier

Parameter	Search space	The best values
learning_rate	[0.02, 0.03, 0.04]	0.03
early_stopping_rounds	[3, 5, 10, 15, 2]	3
n_estimators	[10, 50, 100, 150]	100

3.3.7 Light gradient boosting machine (LightGBM)

LightGBM represents a sophisticated ensemble learning method that uses a leaf-wise splitting strategy, focusing on splitting leaves with the most potential to reduce errors. This approach leads to faster training times and improved accuracy by efficiently identifying and terminating unnecessary branches. As a consequence, LightGBM has flexibility and support for parallel training which makes it a powerful tool for various machine learning tasks [37]. The refined parameters employed for the LightGBM classifier are detailed in Table 7.

4 Experimental results

Experiments were conducted on the Kaggle platform using a machine equipped with an Intel CPU (2.20 GHz, 4 cores) and 30 GB RAM. This study investigates the effectiveness of tree-based ensemble models for detecting malicious PDFs using the Evasive-PDFMal2022 dataset and aims to identify the best-performing model among them. Seven state-of-the-art models (LightGBM, CatBoost, XGBoost, random forest, Extra Trees, HGB, and AdaBoost Classifier) were conducted. A 5-fold Grid Search was utilized for optimizing hyperparameters and assessing model

Table 3 Tuned parameters for histogram-based gradient boosting classifier

Parameter	Search space	The best values
learning_rate	[0.02, 0.03, 0.04]	0.04
max_depth	[3, 5, 8, 13]	5
max_iter	[50, 100, 200, 300]	200
max_leaf_nodes	[5, 8, 13, 21]	8
loss	[log_loss, binary_crossentropy, auto]	auto

Table 6 Tuned parameters for extreme gradient boosting classifier

Parameter	Search space	The best values
Objective	[binary: logitraw, binary: logistic]	binary: logistic
eval_metric	[error, log loss]	logloss
colsample_bytree	[0.6, 0.8, 1.0]	0.6
max_depth	[3, 5, 6, 8]	3
subsample	[0.6, 0.8, 1.0]	0.8
n_estimators	[50, 100, 200]	100

Table 7 Tuned parameters light gradient boosting machine classifier

Parameter	Search space	The best values
learning_rate	[0.05, 0.1, 0.2]	0.2
min_child_samples	[3, 5, 10, 15]	15
min_child_weight	[1e-5, 1e-3, 1e-2, 1e-1]	1e-5
num_leaves	[5, 10, 20]	10
reg_lambda	[1, 5, 10, 20]	5
max_depth	[3, 5, 8, 13, 21]	5

generalization. Models performance is assessed using AUC, accuracy, recall, precision, F1-score, and inference time [38]. SHAP explanation was used for model interpretation. The Wilcoxon Signed Rank test and paired t-test were valuable statistical tests. This section describes each of these points in detail.

4.1 Tree-based ensemble models performance

This section evaluates the performance of tree-based ensemble models on the Evasive-PDFMal2022 dataset to determine the best-performing model and analyze its impact.

4.1.1 The performance of models without optimization

Table 8 presents that the LightGBM model outperforms other tree-based ensemble models without any hyperparameter fine-tuning, achieving the fastest inference time of 0.2995 s and the best performance across all evaluation

metrics. Extra Trees ranks second in terms of AUC, recall, precision, and F1-score. CatBoost and HGB produce identical results, while random forest and XGBoost show comparable performance. CatBoost has the slowest inference time of 5.2177 s, and AdaBoost performs the worst performance in all evaluation metrics.

4.1.2 The performance of models with optimization

Table 9 presents the performance evaluation after incorporating hyperparameter tuning. The LightGBM model achieves an accuracy of 99.95%, recall at 100%, precision at 99.90%, and fast inference time of 0.1723 s. XGBoost and random forest show comparable performance, but random forest has a faster inference time of 0.2590 s. CatBoost, while performing well, has the slowest inference time of 0.9245 s. Surprisingly, AdaBoost's performance deteriorates with hyperparameter fine-tuning.

Table 10 validates the generalization ability of the tree-based ensemble models, with the LightGBM model consistently outperforming the other models, demonstrating its robustness and effectiveness for PDF malware detection on the Evasive-PDFMal2022 dataset.

Figure 2 shows the ROC curves for ensemble classification models during the training step. It is clear that HGB, XGBoost, LightGBM, and random forest achieve perfect ROC curves with AUC scores of 1.00. Moreover, these models are capable of perfectly separating malicious from benign PDFs. Figure 3 presents the AUPRC curves for ensemble classification models during the training step. It is clear that HGB, XGBoost, LightGBM, and random forest all achieved a perfect score of 1.00 on the training

Table 8 Tree-based models performance comparison before optimization

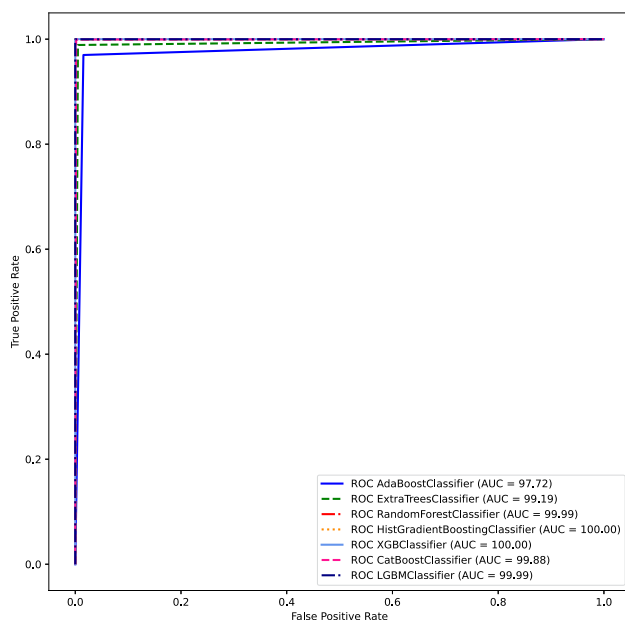
Classifier	AUC	Training-accuracy	Testing-accuracy	Recall	Precision	F1	Inference time
RF	99.16	100.00	99.15	99.16	99.13	99.14	1.028
ET	99.30	100.00	99.30	99.30	99.28	99.29	0.8883
HGB	99.21	100.00	99.20	99.21	99.17	99.19	0.6627
AdaBoost	98.61	99.23	98.60	98.61	98.57	98.59	0.6660
XGBoost	99.17	100.00	99.15	99.17	99.12	99.14	0.9406
CatBoost	99.21	99.96	99.20	99.21	99.17	99.19	5.2177
LightGBM	99.35	100.00	99.35	99.35	99.34	99.34	0.2995

Table 9 Tree-based models performance comparison after optimization

Classifier	AUC	Training-accuracy	Testing-accuracy	Recall	Precision	F1	Inference time
RF	99.90	99.99	99.90	99.90	99.90	99.90	0.4597
ET	98.90	99.23	98.86	98.36	99.51	98.93	0.3121
HGB	99.85	100.00	99.84	99.81	99.90	99.86	0.6205
AdaBoost	97.88	97.79	97.82	97.11	98.82	97.96	0.2881
XGBoost	99.90	99.98	99.90	99.90	99.90	99.90	0.7187
CatBoost	99.68	99.89	99.69	99.81	99.62	99.71	0.9645
LightGBM	99.94	99.99	99.95	100.00	99.90	99.95	0.1723

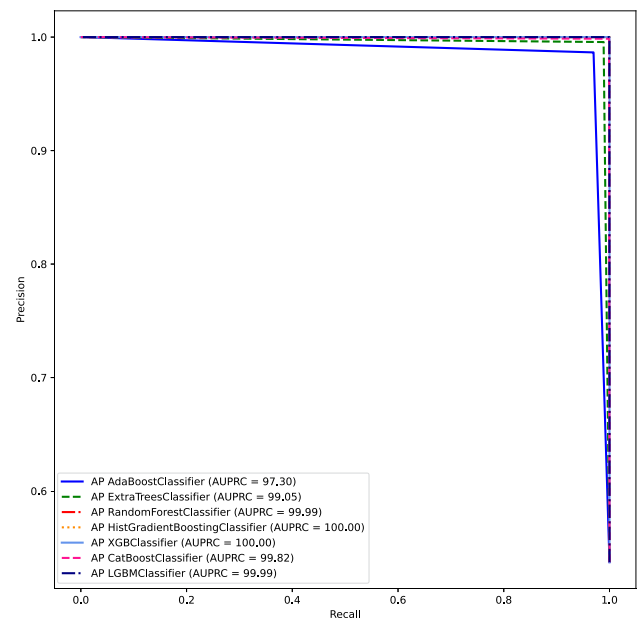
Table 10 Generalization ability of tree-based ensemble models

Classifier	Accuracy (Mean $\pm$ standard deviation)
RF	99.78 $\pm$ 0.08
ET	99.43 $\pm$ 0.17
HGB	99.78 $\pm$ 0.08
AdaBoost	99.66 $\pm$ 0.50
XGBoost	99.75 $\pm$ 0.05
CatBoost	99.69 $\pm$ 0.12
LightGBM	99.82 $\pm$ 0.09

**Fig. 2** The ROC curves for ensemble classification models during training step

data. This demonstrates their remarkable ability to identify true positives (malicious PDFs) while minimizing false positives. Adaboost also performs well, but with a less perfect score of 97% on both ROC and AUPRC.

Figures 4 and 5 illustrate the ROC and AUPRC curves during the testing step. It is clear that LightGBM, HGB, XGBoost, CatBoost, and random forest classifiers perform

**Fig. 3** The AUPRC curves for ensemble classification models during training step

effectively, with ROC and AUPRC scores close to 1.00. Thus, these models efficiently generalized from the training data to the unseen test data. On the other hand, Extra Trees and AdaBoost show a small decrease in performance, indicating that they may be less effective at detecting new malware variations.

Figure 6 shows the confusion matrices of tree-based ensemble models after applying feature engineering and hyperparameter tuning. LightGBM demonstrates exceptional performance, achieving high true positive samples of 1037 with one false positive and zero false negative samples, making it the most accurate model. XGBoost shows strong performance and also has the same values as the random forest confusion matrix of 1036 true positive samples with one false positive sample, one false negative, and 891 true negatives. HGB and CatBoost misclassify 2 samples as false negatives, and ExtraTrees misclassify 17 samples as false negatives. AdaBoost exhibits the highest misclassifying model with 30 samples as false negatives

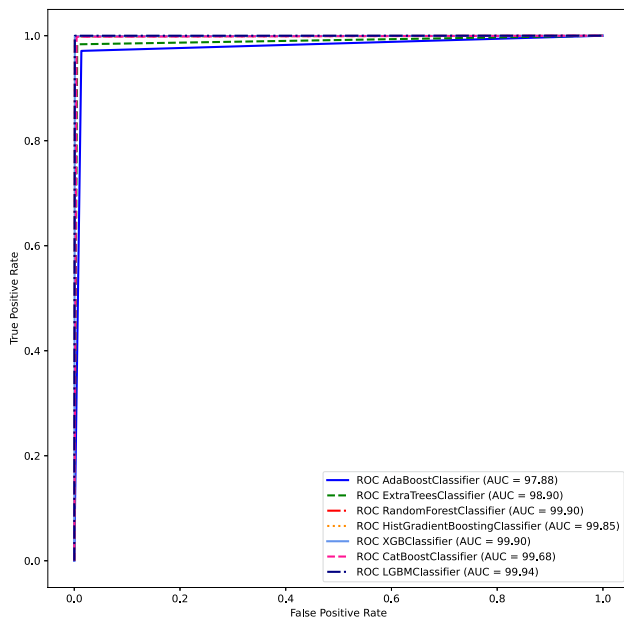


Fig. 4 The ROC curves for ensemble classification models during testing step

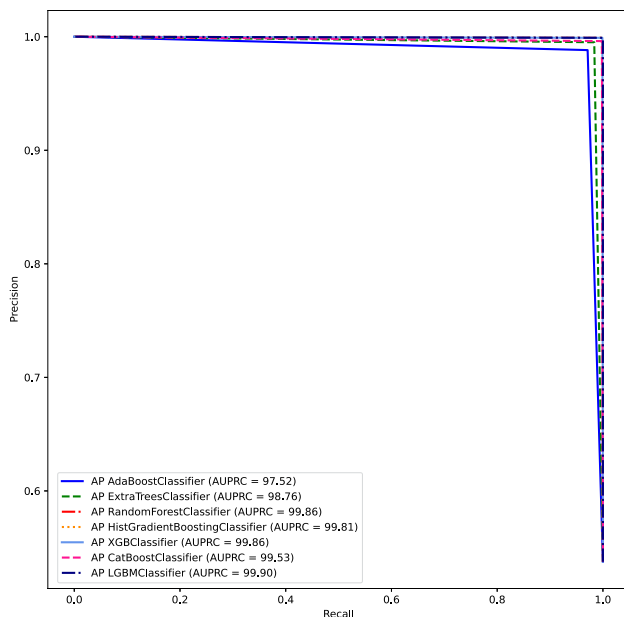


Fig. 5 The AUPRC curves for ensemble classification models during testing step

and 12 samples as false positives, indicating a higher rate of missed detections and false alarms.

After comparing the performance of multiple tree-based ensemble models, we selected the best-performing classifier (LightGBM) for further analysis. This analysis included tuning hyperparameters and employing SHAP for model interpretability.

Figure 7 highlights the importance of different hyperparameters for the LightGBM model. The “num_leaves”,

accounting for approximately 88% of the total importance, has the most significant impact on model performance. This finding underscores the crucial role of tuning the complexity of decision trees to achieve optimal accuracy and generalization.

4.1.3 The performance of models on the validation data set

The main goal of this study is to develop a model capable of handling unseen samples during the training step. To further assess the generalization ability of the proposed model, the model is tested using new data collected from diverse sources. The provided examples consist of 1000 PDF files that are benign and 55 PDF files that are malicious. This dataset provided an independent evaluation of the approach beyond the initial training and testing phases. The validation dataset used in this study can be accessed through the Github website.¹

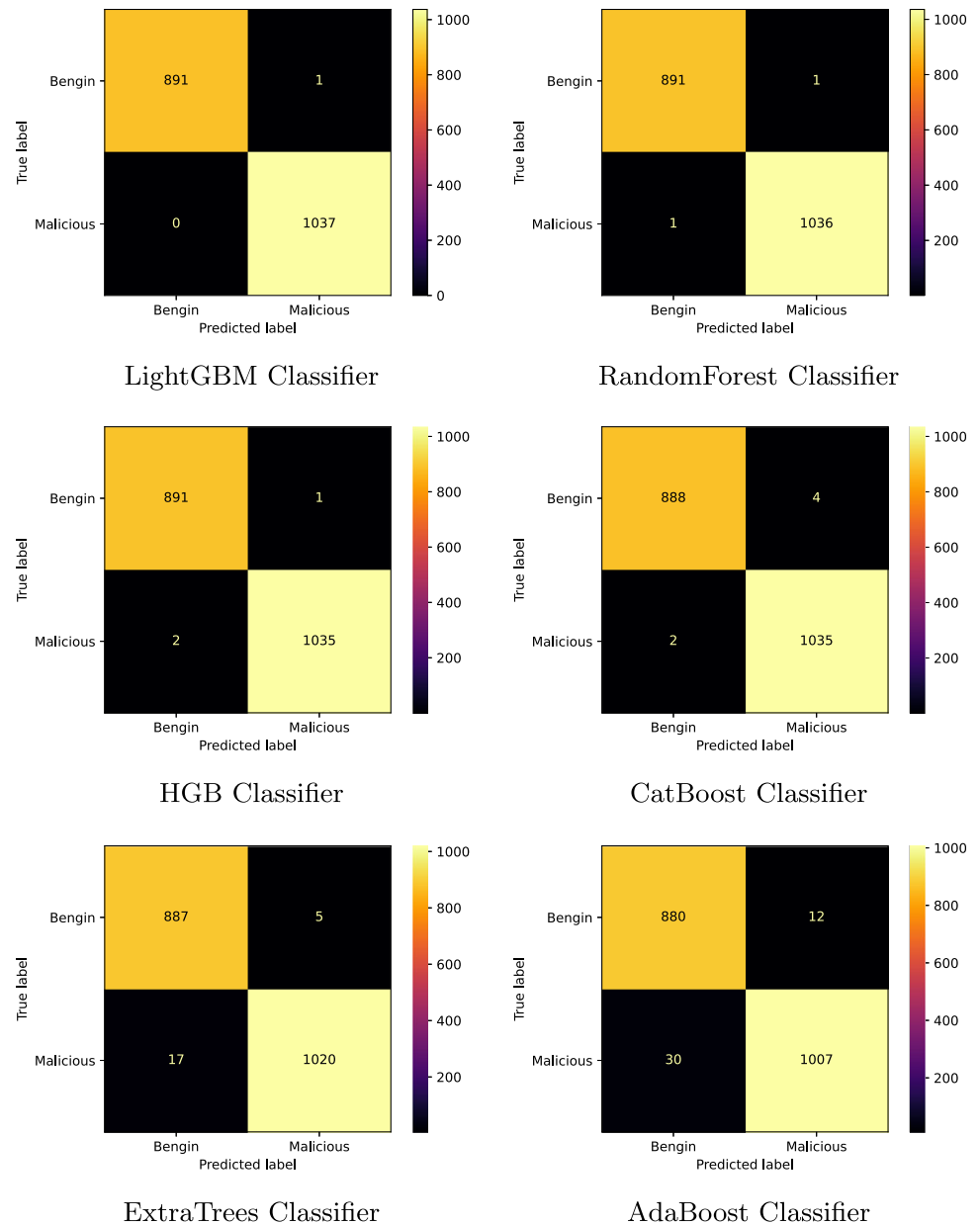
As shown in Table 11, LightGBM demonstrated exceptional performance on the validation dataset, achieving an AUC of 98.91% and a recall of 98.18%. These results highlight its effectiveness in minimizing false negatives, ensuring that the model accurately identifies malicious PDFs. Furthermore, XGBoost showcased a strong ability to minimize false alarms, attaining a precision of 96.36%. This indicates that XGBoost is proficient at avoiding misclassifying benign files as malicious, contributing to a reduced number of false positives.

4.2 Model interpretations with SHAP

This study employs a tree-based SHAP explanation for model interpretability due to its computational efficiency. Tree-based SHAP provides an accurate approximation of Shapley values. This trade-off between speed and absolute precision makes tree-based SHAP ideal for interpreting tree-based models [39].

The shapely values were calculated for the test set of the Evasive-PDFMal2022 dataset, which was trained using the LightGBM classifier. Figure 8 presents the classical LightGBM feature importance, while Fig. 9 shows the SHAP bar plot, which represents the mean of the absolute SHAP values. Both plots rank “text” and “header” as the most influential feature. SHAP provides valuable insights into how features impact the model’s predictions, indicating both the magnitude and direction of their influence. This added detail is crucial for understanding the model’s decision-making process.

¹ https://github.com/kartik2309/Malicious_pdf_detection.

Fig. 6 Confusion matrices after tree-based ensemble models optimization

4.2.1 Shapely global features interpretation

The SHAP summary plot calculates the global importance score and provides individual prediction explanations by averaging the absolute Shapley values per feature across the dataset. This approach facilitates a comprehensive visual summary, as shown in Fig. 10. Each sample is color-coded based on its relative abundance of PDFs, with blue indicating low abundance and red indicating high abundance.

The PDF features are arranged in the figure in descending order based on their relative importance: $\sum_{j=1}^N |\phi_i^{(j)}|$, where N is the total number of samples, and ϕ_i is the Shapley value of feature i . Each dot in the summary

plot represents a sample plotted against its impact on the model output $\phi_i^{(j)}$. The “text” feature shows a strong positive influence on detecting malicious PDFs. In contrast, the “pdf size” feature exhibits a negative correlation, suggesting smaller files are more likely to be benign.

Figure 11 shows the relationship between the “text” feature and the SHAP values attributed to it for each PDF file. Higher SHAP values indicate a greater likelihood of maliciousness due to text features. The presence of a header, when the text size increases up to 1.0, further elevates the predicted risk of maliciousness.

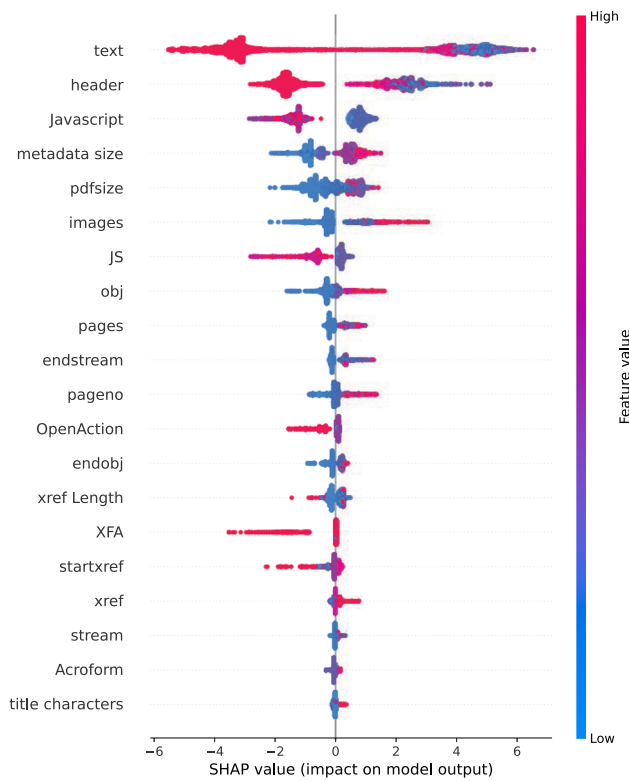


Fig. 10 SHAP summary plot for the LightGBM model

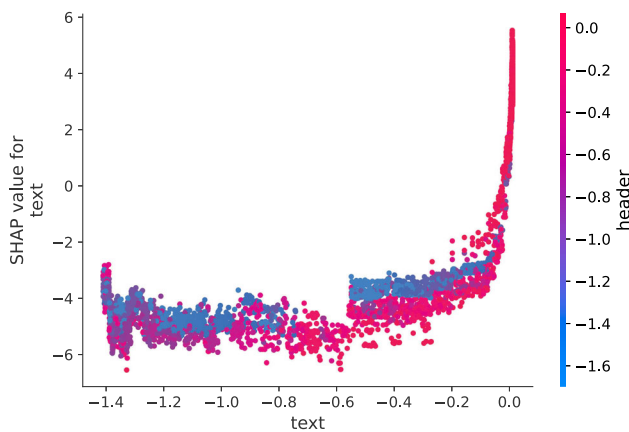


Fig. 11 SHAP dependence plots for the LightGBM model interaction effect between PDF parameters

prediction, while blue bars represent features contributing to a benign prediction.

In Fig. 12a, which shows a malicious prediction with a probability output of 9.676, “text” and “header” remain significant features, aligning with the global importance rankings. In contrast, Fig. 12b, which displays a benign prediction with a probability output of −9.787, highlights the importance of low impacts for “text” and “header” in driving this classification. While “metadata size” leans towards a malicious prediction, its influence is outweighed

by other features that collectively contribute to the benign prediction.

4.3 Comparative analysis of statistical tests

The Wilcoxon signed-rank test and Mann–Whitney U test were employed to determine the statistical significance of the performance improvements observed before hyperparameter tuning (Table 8) and after hyperparameter tuning (Table 9). Table 12 presents the testing accuracy values for each model, comparing performance before and after optimization. These tests provide a robust evaluation of the significance of the observed improvements, allowing us to draw meaningful conclusions regarding the impact of feature engineering and hyperparameter tuning on model performance.

The null hypothesis (H_0) assumes that there is no significant difference between the 2 groups. Therefore, in this study, the null hypothesis (H_0) assumes that the mean testing accuracy after optimization (μ_{after}) is greater than or equal to the mean testing accuracy before optimization (μ_{before}), denoted as $H_0 : \mu_{\text{after}} \geq \mu_{\text{before}}$. On the other hand, the alternative hypothesis (H_1) proposes that the mean testing accuracy after optimization is less than the mean testing accuracy before optimization, expressed as $H_1 : \mu_{\text{after}} \leq \mu_{\text{before}}$.

4.3.1 The Wilcoxon signed rank test (WSR)

The WSR Test, a non-parametric alternative to the paired t-test [40], is employed to assess the presence of a significant difference between 2 population means. This test is particularly suitable when the distribution of differences between paired observations significantly deviates from normality. The formula for the WSR test is as follows:

$$Z = \frac{W - \mu}{\sigma} \quad (2)$$

To apply the WSR test, we obtain and rank the differences between the before and after test accuracy based on the sample of all models, as presented in Table 13. The test statistic, denoted as W , corresponds to the smaller value between the absolute sums of positive ranks ($W+$) and negative ranks ($W-$). Table 13 illustrates the stepwise procedure for calculating $W-$ and $W+$.

To calculate the minimum value, W , resulting in $W = \text{Min}(W-, W+) = \text{Min}(7, 21) = 7$. Next, we determine the population mean, μ , using the equation $\mu = \frac{n(n+1)}{4}$, where n represents the sample size. Applying this equation with $n = 7$, we obtain $\mu = \frac{n(n+1)}{4} = \frac{7(7+1)}{4} = 14$. Furthermore, we calculate the population variance, σ^2 , using the equation $\sigma^2 = \frac{n(n+1)(2n+1)}{24}$. Substituting the value of n as 7, we find

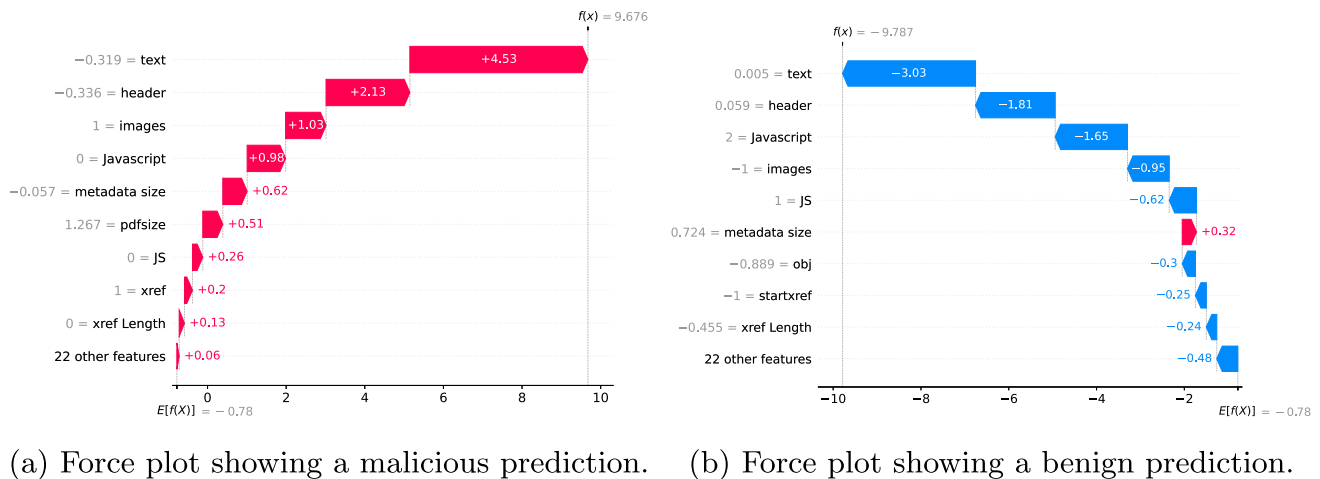


Fig. 12 Local explanations using Shapley values with random samples

Table 12 The testing accuracy for each model

Model	After optimization	Before optimization
RF	99.90	99.15
ET	98.86	99.30
HGB	99.84	99.20
AdaBoost	97.82	98.60
XGBoost	99.90	99.15
CatBoost	99.69	99.20
LightGBM	99.95	99.35

$\sigma^2 = \frac{n(n+1)(2n+1)}{24} = \frac{7(7+1)(2*7+1)}{24} = 35$. To approximate the distribution of the “After” scenario, we consider the normal distribution. We set $W+$ as 21, and state that $W+$ follows a normal distribution with mean μ and variance σ^2 , denoted as $W+ \sim N(14, 35)$. We then calculate the z-score, Z , using the formula $Z = ((W+) - \mu) / \sqrt{\sigma^2}$. Substituting the given values, we find $Z = \frac{21-14}{\sqrt{35}} = 1.18$.

Table 13 Steps of the Wilcoxon signed rank test for calculating $W-$ and $W+$

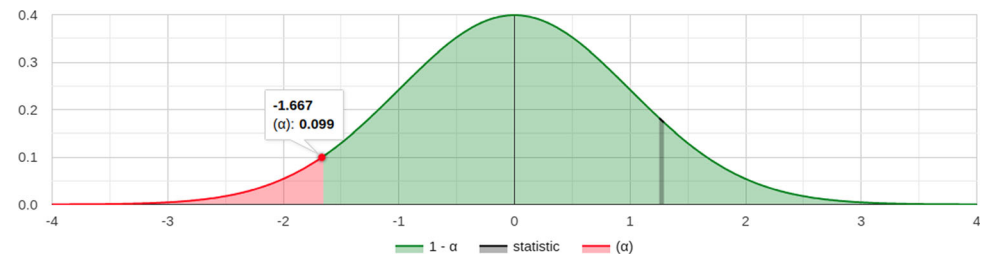
Before	After	Difference	Abs diff	Sign	Rank - Abs diff	W-	W+
99.15	99.9	0.75	0.75	+	5.5		5.5
99.3	99.86	0.56	0.56	+	2		2
99.2	99.84	0.64	0.64	+	4		4
99.6	97.82	-1.78	1.78	-	7	7	
99.15	99.9	0.75	0.75	+	5.5		5.5
99.2	99.69	0.49	0.49	+	1		1
99.35	99.95	0.6	0.6	+	3		3
Total						7	21

Finally, we calculate $p - value = p = p(x \leq 1.18) = 0.898$.

The test statistic Z is equal to 1.27, falling within the 95% acceptance region: $[-1.6449, \infty]$ as shown in Fig. 13. Since the p -value ($p = 0.898$) is greater than the significance level ($\alpha = 0.01$), the null hypothesis (H_0) cannot be rejected. In other words, the obtained results provide support for H_0 . Therefore, based on the WSR Test, we conclude that there is a statistically significant difference in the medians between the before and after scenarios (Table 14).

4.3.2 Mann–Whitney U test

The Mann–Whitney U test, a nonparametric statistical test, was employed to investigate potential variations in performance among the different models [41]. Table 15 presents the results of the Mann–Whitney U test. The table displays the p-values representing the statistical comparisons between different models. The diagonal entries, denoted by “-”, indicate self-comparisons and are

Fig. 13 The p -value of the WSR Test**Table 14** The parameters of the Wilcoxon signed rank test

Test parameter	Value	Test parameter	Value
SD of differences (Sd)	0.9165	α	0.01
Average of differences ($\bar{x}_d$)	0.2871	P -value	0.898
Number of pairs (N/n)	7	σ^2	35
$W = \min(W^-, W^+)$	$\min(7, 21) = 7$	μ	14
Test statistic Z	1.27	Critical value	-1.6449

Table 15 Results of Mann Whitney U test (after optimization)

	RF	ET	HGB	AdaBoost	XGBoost	CatBoost	LightGBM
RF	-	0.991	0.87	0.991	0.5	0.987	0.104
ET		-	0.009	0.987	0.009	0.009	0.013
HGB			-	0.991	0.104	0.951	0.082
AdaBoost				-	0.009	0.009	0.013
XGBoost					-	0.987	0.082
CatBoost						-	0.013
LightGBM							-

not applicable. The values in the table represent the degree of statistical difference between the models, wherein lower values suggest greater dissimilarity.

The outcomes indicate that, except for the AdaBoost technique, there is not a large difference between the 2 comparisons of the models. Consequently, the differences in the model's performance were relatively minor. Therefore, it is more important to go for the model with better performance to get better results. Finally, the LightGBM model demonstrated the highest performance and yielded the best results.

5 Discussion

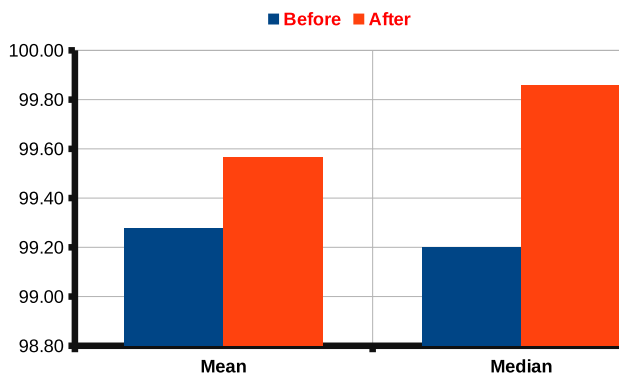
This study highlights the effectiveness of a LightGBM model for detecting malicious PDF files (PDFMal) on the challenging Evasive-PDFMal2022 dataset. Rigorous data preparation and hyperparameter tuning using a 5-fold cross-validation grid search were crucial for optimizing the model's performance. LightGBM classifier achieved

perfect recall (100%) and zero false negatives (Table 9), ensuring that no malicious files were overlooked. The test ROC curve shows that the LightGBM model generalizes exceptionally well, achieving an AUC score close to 1.00. This means it effectively identifies malicious PDFs even when encountering previously unseen examples (Fig. 4). The AUPRC curve (Fig. 5) confirms its high performance, demonstrating strong precision and recall. This robust performance is crucial for handling real-world PDF malware detection, especially against evolving and potentially evasive threats.

Table 16 presents a comparison of the performance of the LightGBM model with state-of-the-art models on the Evasive-PDFMal2022 dataset. The results demonstrate that the LightGBM approach surpasses existing machine learning methods [3, 13, 25] and deep learning approach [26] in terms of both false negative rate and false positive rate. Additionally, the LightGBM approach also exhibits a faster inference time of 0.1723 s. The model's exceptional performance, particularly its ability to minimize false negatives, is crucial in cybersecurity, where missing

Table 16 Comparative analysis of the proposed model and existing models on the Evasive-PDFMal2022 Dataset

Model	AUC	Accuracy	Recall	Precision	F1	Inference time
Maryam et al. [3]	–	98.69	98.87	98.88	98.77	–
Abu Al-Haija et al. [13]	–	98.84	98.90	98.80	98.80	2.174
Ekholm et al. [25]	–	98.90	98.90	99.00	98.9	–
Chandran et al. [26]	98.77	98.20	94.43	98.65	96.40	–
The proposed model	99.94	99.95	100.00	99.90	99.95	0.1723

**Fig. 14** The averages and medians of data

malicious files can have severe consequences. LightGBM effectively minimized false alarms, achieving a precision of 99.90% and demonstrating high accuracy (99.95%).

The SHAP explanations explain the LightGBM predictions (Fig. 10) to enhance interpretability and user trust. This analysis revealed that the “text” and “header” features are strong indicators of malicious PDFs (Fig. 12), aligning with the understanding that malicious PDFs often use obfuscated text or exploit vulnerabilities in header information.

Mean and median analyses were also conducted to compare the central tendencies of data between the 2 groups. The WSR Test indicated a statistically significant difference in medians between the “Before” scenario (Mean = 99.28, Median = 99.2, $n = 7$) and the “After” scenario (Mean = 99.57, Median = 99.86, $n = 7$), as illustrated in Fig. 14. As shown in Fig. 14 compares the mean and median values of the measured variable before and after the intervention, highlighting the substantial improvement achieved through the intervention.

The Mann–Whitney U test, WSR test, and mean and median analyses were valuable statistical tools in this research. Each test served a specific purpose and contributed unique insights to the analysis. Notably, the WSR Test exhibited robustness when the assumption of normality was violated, making it an advantageous non-parametric alternative to the paired t-test. While this study demonstrates the effectiveness of the tree-based ensemble model especially the LightGBM model for detecting malicious PDFs, it’s important to acknowledge some

limitations. The Evasive-PDFMal2022 dataset, while valuable, may not fully encompass the diverse range of PDF malware threats, and collecting more data, especially new and evasive examples, can be a significant challenge. Furthermore, the framework is currently limited to detecting malicious PDFs, and expanding its capabilities to other document types, such as .docx files, is a critical area for future research.

6 Conclusion

This study presents a robust framework for detecting malicious PDF documents (PDFMal) using tree-based ensemble models. LightGBM outperformed other tree-based ensemble models on the challenging Evasive-PDFMal2022 dataset, achieving 99.95% accuracy, 100% recall, an F1-score of 99.95%, and a fast prediction time of 0.1723 s, making it appropriate for real-time applications. Furthermore, the paired t-test and the WSR test illustrate statistical differences between tree-based ensemble models before and after optimization. Additionally, the SHAP technique is clarified to provide explanations for the model’s predictions. The proposed framework effectively detects malicious PDF files, but it is currently limited to PDF formats. Future research should expand the model’s capabilities to encompass other document types, such as .docx files, to create a more comprehensive and robust solution for detecting malicious documents. Additionally, further investigation into emerging evasion techniques and their impact on model performance is necessary to ensure continued effectiveness.

Acknowledgements Not applicable.

Author contributions Conceptualization: Ibrahim Gad and Mona Elattar. Data curation: Mona Elattar. Formal Analysis: Ibrahim Gad, and Ahmed Younes. Investigation: Ahmed Younes, and Islam Elkabani. Methodology: Ibrahim Gad and Islam Elkabani. Software: Mona Elattar. Validation: Mona Elattar. Visualization: Mona Elattar, and Ibrahim Gad. Writing original draft, Mona Elattar. Writing, review, and editing: Ahmed Younes, Ibrahim Gad, and Islam Elkabani. Supervision: Islam Elkabani, and Ahmed Younes. Senior Administration: Ahmed Younes.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB).

Data availability The data that support the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare no conflict of interest.

Ethical approval and consent to participate Not applicable.

Consent to publish Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Fleury N, Dubrunquez T, Alouani I (2021) Pdf-malware: an overview on threats, detection and evasion attacks. *ArXiv abs/2107.12873*
2. Liu D, Wang H, Stavrou A (2014) Detecting malicious javascript in pdf through document instrumentation. In: 2014 44th Annual IEEE/IFIP international conference on dependable systems and networks, pp. 100–111. <https://doi.org/10.1109/DSN.2014.92>
3. Issakhani M, Victor P, Tekeoglu A, Habibi Lashkari A (2022) Pdf malware detection based on stacking learning, pp. 562–570. <https://doi.org/10.5220/0010908400003120>
4. Jeong Y-S, Woo J, Kang AR (2019) Malware detection on byte streams of pdf files using convolutional neural networks. *Secur Commun Netw* 2019:1–9. <https://doi.org/10.1155/2019/8485365>
5. Sudhakar Kumar S (2020) An emerging threat fileless malware: a survey and research challenges. *Cybersecurity*. <https://doi.org/10.1186/s42400-019-0043-x>
6. Rawal BS, Manogaran G, Peter A (2023) Effective cybersecurity. Springer, Singapore, pp 87–102
7. Alsmadi T, Alqudah N (2021) A survey on malware detection techniques. In: 2021 International conference on information technology (ICIT), pp. 371–376. <https://doi.org/10.1109/ICIT52682.2021.9491765>
8. Souri A, Hosseini R (2018) A state-of-the-art survey of malware detection approaches using data mining techniques. *Hum Centric Comput Inf Sci*. <https://doi.org/10.1186/s13673-018-0125-x>
9. Al-Marghilani A (2021) Comprehensive analysis of iot malware evasion techniques. *Eng Technol Appl Sci Res* 11(4), 7495–7500 <https://doi.org/10.48084/etasr.4296>
10. Doreswamy Hooshmand MK, Gad I (2020) Feature selection approach using ensemble learning for network anomaly detection. *CAAI Trans Intell Technol* 5(4):283–293. <https://doi.org/10.1049/trit.2020.0073>
11. Chakir O, Rehaimi A, Sadqi Y, Abdellaoui Alaoui EA, Krichen M, Gaba GS, Gurtov A (2023) An empirical assessment of ensemble methods and traditional machine learning techniques for web-based attack detection in industry 5.0. *J King Saud Univ Comput Inf Sci* 35(3):103–119. <https://doi.org/10.1016/j.jksuci.2023.02.009>
12. Zelinka I, Amer E (2019) An ensemble-based malware detection model using minimum feature set. *Mendel* 25(2), 1–10. <https://doi.org/10.13164/mendel.2019.2.001>
13. Abu Al-Haija Q, Odeh A, Qattout H (2022) Pdf malware detection based on optimizable decision trees. *Electronics* 11(19):3142. <https://doi.org/10.3390/electronics11193142>
14. Gao X, Shan C, Hu C, Niu Z, Liu Z (2019) An adaptive ensemble machine learning model for intrusion detection. *IEEE Access* 7:82512–82521. <https://doi.org/10.1109/access.2019.2923640>
15. Ye A, Wang Z (2022) Tree-based deep learning approaches. *Modern deep learning for tabular data*, 549–598. https://doi.org/10.1007/978-1-4842-8692-0_7
16. Sami A, Yadegari B, Rahimi H, Peiravian N, Hashemi S, Hamze A (2010) Malware detection based on mining api calls. In: Proceedings of the 2010 ACM symposium on applied computing, pp. 1020–1025. <https://doi.org/10.1145/1774088.1774303>
17. Matloob F, Ghazal TM, Taleb N, Aftab S, Ahmad M, Khan MA, Abbas S, Soomro TR (2021) Software defect prediction using ensemble learning: a systematic literature review. *IEEE Access* 9:98754–98771. <https://doi.org/10.1109/access.2021.3095559>
18. Taha AA, Malebary SJ (2020) An intelligent approach to credit card fraud detection using an optimized light gradient boosting machine. *IEEE Access* 8:25579–25587. <https://doi.org/10.1109/access.2020.2971354>
19. Alqahtani EJ, Zagrouba R, Almuhaideb A (2019) A survey on android malware detection techniques using machine learning algorithms. In: 2019 Sixth international conference on software defined systems (SDS), pp. 110–117. <https://doi.org/10.1109/SDS.2019.8768729>
20. zollak: Malware dump. <https://contagiodump.blogspot.com/>
21. Issakhani M, Victor P, Tekeoglu A, Lashkari A (2022) CIC-Evasive-PDFMal2022 datasets: Canadian institute for cybersecurity. University of new Brunswick in Fredericton. <https://www.unb.ca/cic/datasets/pdfmal-2022.html>
22. Falah A, Pan L, Huda S, Pokhrel SR, Anwar A (2021) Improving malicious pdf classifier with feature engineering: a data-driven approach. *Futur Gener Comput Syst* 115:314–326. <https://doi.org/10.1016/j.future.2020.09.015>
23. Yoo S, Kim S, Kim S, Kang BB (2021) Ai-hydra: advanced hybrid approach using random forest and deep learning for malware classification. *Inf Sci* 546:420–435. <https://doi.org/10.1016/j.ins.2020.08.082>
24. Lu X, Wang F, Jiang C, Lio P (2021) A universal malicious documents static detection framework based on feature generalization. *Appl Sci* 11(24):12134. <https://doi.org/10.3390/app112412134>
25. EKHOLOM O (2022) Increased evasion resilience in modern PDF malware detectors: Using a more evasive training dataset. KTH Royal Institute of Technology. <http://www.diva-portal.org/smash/get/diva2:1678561/FULLTEXT01.pdf>
26. Chandran PP, Hema RN, Jeyakarthish M (2022) Invasive weed optimization with stacked long short term memory for PDF malware detection and classification. *Int J Health Sci*. <https://doi.org/10.53730/ijhs.v6ns5.9540>
27. Myers TA (2011) Goodbye, listwise deletion: presenting hot deck imputation as an easy and effective tool for handling missing data. *Commun Methods Meas* 5(4):297–310. <https://doi.org/10.1080/19312458.2011.624490>
28. Ostroumova L, Gusev G, Vorobev A, Dorogush AV, Gulina A (2017) Catboost: unbiased boosting with categorical features. In:

- Neural information processing systems. <https://api.semanticscholar.org/CorpusID:5044218>
29. Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) Smote: synthetic minority over-sampling technique. *J Artif Intell Res* 16:321–357. <https://doi.org/10.1613/jair.953>
 30. Liashchynskiy PB, Liashchynskiy P (2019) Grid search, random search, genetic algorithm: a big comparison for nas. *ArXiv abs/1912.06059*
 31. Arangala C (2023) Linear algebra with machine learning and data. Chapman and Hall/CRC, New York. <https://doi.org/10.1201/9781003025672>
 32. Geurts P, Ernst D, Wehenkel L (2006) Extremely randomized trees. *Mach Learn* 63(1):3–42. <https://doi.org/10.1007/s10994-006-6226-1>
 33. Guryanov A (2019) Histogram-based algorithm for building gradient boosting ensembles of piecewise linear decision trees. In: *Analysis of images, social networks and texts: 8th International conference, AIST 2019, Kazan, Russia, July 17–19, 2019, Revised selected papers 8*, pp. 39–50. Springer
 34. Freund Y (1995) Boosting a weak learning algorithm by majority. *Inf Comput* 121(2):256–285. <https://doi.org/10.1006/inco.1995.1136>
 35. Dorogush AV, Ershov V, Gulin A (2018) Catboost: gradient boosting with categorical features support. *ArXiv abs/1810.11363*
 36. Chen T, Guestrin C (2016) Xgboost: a scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 785–794. Association for computing machinery, New York, NY, USA. <https://doi.org/10.1145/2939672.2939785>
 37. Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu T-Y (2017) Lightgbm: a highly efficient gradient boosting decision tree. In: *Neural information processing systems*. <https://api.semanticscholar.org/CorpusID:3815895>
 38. Gad I, Elmezain M, Alwateer MM, Almaliki M, Elmarhomy G, Atlam E (2023) Breast cancer diagnosis using a machine learning model and swarm intelligence approach. In: *2023 1st International conference on advanced innovations in smart cities (ICAISC)*, pp. 1–5. <https://doi.org/10.1109/ICAISC56366.2023.10085393>
 39. Lundberg SM, Erion GG, Lee S-I (2018) Consistent individualized feature attribution for tree ensembles. *ArXiv abs/1802.03888*
 40. Sheskin DJ (2020) Handbook of parametric and nonparametric statistical procedures. Chapman and hall/CRC, New York. <https://doi.org/10.1201/9780429186196>
 41. Su L, Hounye AH, Pan Q, Miao K, Wang J, Hou M, Xiong L (2024) Explainable cancer factors discovery: shapley additive explanation for machine learning models demonstrates the best practices in the case of pancreatic cancer. *Pancreatology* 24(3):404–423. <https://doi.org/10.1016/j.pan.2024.02.002>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.